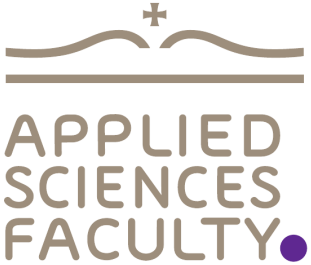
C++ — лекція 23. Стандартні контейнери





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Стандартна бібліотека C++ надає ряд корисних допоміжних засобів, що безпосередньо стосуються STL.

- Шаблонні функції min(x, y) та max(x,y), які повертають, відповідно менше та більше із двох переданих значень, <algorithm>.
- Шаблонна функція swap(a1,a2), яка міняє аргументи місцями, оголошена так (в C++11 оголошення дещо відрізняється, див. документацію):

  ```
  template<class T>
   inline  void swap(T& a, T& b);
  ```

  C++98: <algorithm>, C++11: <utility>.
- Шаблон пари, pair<T1, T2>, для збереження двох різнотипних значень, <utility>. Детальніше — див. далі.
- Шаблон numeric_limits, з файлу <limits>, для представлення інформації про типи, спеціалізований для всіх фундаментальних типів. Детальніше — див. далі.
- Простір імен `std::rel_ops`, <utility>, див. далі.

std::pair оголошено якось так:

```
template <class T1, class T2>
        struct pair {
            //Типи першого та другого аргументів
            typedef T1 first_type;
            typedef T2 second_type;
            // Члени
            T1 first;
            T2 second;

             // Конструктор по замовчуванню, який викликає конструктори
             // по замовчуванню для T1 і T2
            pair(): first(T1()), second(T2()) {}

            //Конструктор із двома аргументами
            pair(const T1& a, const T2& b): first(a), second(b) {}

            // Конструктор копіювання із неявним перетворенням
            template<class U, class V>
            pair(const pair<U,V>& p): first(p.first), second(p.second) {}
        };
```

Також для std::pair існує ряд допоміжних функцій:

```
// Оператори порівняння
template <class T1, class T2>
bool operator== (const pair<T1,T2>&, const pair<T1,T2>&);

template <class T1, class T2>
bool operator< (const pair<T1,T2>&, const pair<T1,T2>&);

// Аналогічно: !=, <=, >, >=

// Функція для створення пар
// Зручна завдяки здатності виводити тип аргуметнів
template <class T1, class T2>
pair<T1,T2> make_pair (const T1&, const T2&);
```

- C++11 додає метод pair::swap і відповідне перевантаження std::swap для pair, та засоби взаємодії пар із кортежами tuple.
- Ще одна новинка – тег `std::piecewise_construct_t`. Коли конструктору передані кортежі, дозволяє визначити – вставляти їх, чи передати поелементно конструктору. Детальніше див. `https://en.cppreference.com/w/cpp/utility/piecewise_construct_t`.
- Трішки про реалізацію: `https://devblogs.microsoft.com/oldnewthing/20230801-00/?p=108509` – "Inside STL: The pair and the compressed pair".

Приклади

```
pair<int,double> p1;
p1. first  = 5;
p1.second = 3.14;

if (p1 == make_pair(5,3.0))
{
    cout << "Something wrong!" << endl;
}
```

Увага: всі оператори порівняння порівнюють спершу first обох аргументів, якщо цього достатньо для однозначної відповіді, second не порівнюється.

numeric_limits

Спеціалізації шаблону numeric_limits, оголошеного в <limits>, зберігає інформацію про різні властивості типів, зокрема ті, що в C зберігаються в файлах <climits> (<limits.h>) і <cfloat> (<float.h>).

Приклад:

```
cout << "max int: " << numeric_limits<int>::max() << endl;
cout << "digits for int: " << numeric_limits<int>::digits << endl;
cout << "decimal digits for int: " << numeric_limits<int>::digits10 <<
```

Результат:

```
max int: 2147483647
digits for int: 31
decimal digits for int: 9
```

Для своїх числових типів можна оголошувати свої спеціалізації numeric_limits.

Деякі константи, оголошені в numeric_limits

- `is_specialized` — true, якщо для переданого аргументу шаблон спеціалізовано, є відповідна інформація. Якщо false, всі поля нижче позбавлені сенсу.
- `is_signed` — знаковий тип.
- `is_integer` — цілий тип.
- `is_exact` — тип є точним, без похибок заокруглення.
- `min()` — мінімальне скінчене[1] значення.
- `max()` — мінімальне значення.
- `digits`, `digits10` — кількість цифр у внутрішньому представленні (зазвичай – двійкових, зберігається у полі `radix`) та десяткових цифр.
- `epsilon()` — віддаль між 1.0 і найближчим більшим за 1.0 числом.
- `round_error()` — міра найбільшої похибки заокруглення.
- і багато інших, див. документацію: `http://www.cplusplus.com/reference/limits/numeric_limits/`.

[1] Деякі типи, включаючи double, вміють представляти нескінчені значення.

std::rel_ops

- У цьому просторі імен оголошуються всі оператори порівняння, реалізовані з використанням лише операторами == і !=. Його включення дозволяє не писати всі оператори порівняння вручну.

```
namespace rel_ops {
  template <class T> bool operator!= (const T& x, const T& y)
                                     { return !(x==y); }
  template <class T> bool operator>  (const T& x, const T& y)
                                     { return y<x; }
  template <class T> bool operator<= (const T& x, const T& y)
                                     { return !(y<x); }
  template <class T> bool operator>= (const T& x, const T& y)
                                     { return !(x<y); }
}
```

<utility> в C++11

- Даний файл заголовків в C++11 містить функцію move(), яка повертає rvalue reference для явного використання переміщення, та деякі інші інструменти, пов'язані із форвардингом та переміщенням.

STL в C++98 містить наступні контейнери:

- vector — динамічний масив, неперервний в пам'яті, швидке додавання в кінець.
- deque — двостороння черга, швидке додавання в кінець та початок.
- list — двозв'язний динамічний список.
- set/multiset — множина, без/із дублікатами елементів, відповідно.
- map/multimap — відображення, без/із дублікатами ключів.
- Крім того, присутні адаптери контейнерів, stack, queue, priority_queue, котрі надають інтерфейс, відповідно, стеку, черги, черги із пріоритетами.

C++11 надає, додатково:

- array — контейнер фіксованого розміру.
- forward_list — однозв'язний список.
- unordered_set/unordered_multiset — хешовані множини (часто швидші, ніж аналогічні set/multiset).
- unordered_map/unordered_multimap — хешовані відображення.

Назви unordered_XXX вжиті, так як hash_XXX широко використовувалося нестандартними розширеннями, їх використання в стандарті породило б багато конфліктів.

"Майже контейнери"

Існує ряд контейнерів, які схожі на контейнери STL, але мають суттєві відмінності від них:

- vector<bool> – спеціалізація шаблону vector, котра зберігає значення типу bool в окремих бітах. Детальніше вона обговорюється в лекції про vector.
- bitset — масив бітів, фіксованого розміру. (boost містить також dynamic_bitset.)
- valarray — класи для математичних обчислень. При всій перспективності, ідея не була доведена до кінця, використовувати їх не варто. Кращою альтернативою буде blitz++ (Увага! не зважаючи на певну закинутість сайту, бібліотека продовжує розвиватися, щоправда, слід використовувати останню версію із системи контролю версій: `http://blitz.hg.sourceforge.net:8000/hgroot/blitz/blitz` — Mercurial), або щось типу Eigen (`http://eigen.tuxfamily.org/`), хоч у цієї бібліотеки первинним призначенням є матричні та векторні обчислення.

"Майже контейнери" — C++11

C++11 додав кілька цікавих конструкцій:

- Кортежі, tuple — як узагальнення pair, для збереження кількох різнотипних змінних.
- initializer_list — списки ініціалізації, які можна використати, щоб надати можливість ініціалізувати користувацькі класи так само, як вбудовані масиви та структури:

```
class MyArray {.....};

MyArray c = {1, 2, 3, 4};
```

Спільні властивості контейнерів

Всі контейнери/для всіх контейнерів:

- Оголошено у однойменному файлі заголовків (<vector>, <deque>, і т.д.).
- Мають конструктори по замовчуванню та конструктори копіювання, деструктори, оператори присвоєння.
- Мають стандартний набір оголошень типів, див. далі.
- Мають методи для отримання ітераторів на початок та кінець: begin(), end(). C++11 додала cbegin(), cend(), котрі повертають відповідні константні ітератори.
- (Крім однонаправленого forward_list та unordered_XXX), мають методи отримання зворотних (reverse) ітераторів: rbegin(), rend(), C++11 додала crbegin(), crend().
- (Крім array, який має фіксований розмір), використовують алокатори для маніпуляції пам'яттю. Детальніше алокатори в цьому курсі (поки) не розглядаються.
- Реалізовано операції == та !=.
- (Крім хешованих, unordered_XXX), реалізовано операції співвідношення — >, >=, <, <=, які працюють лексикографічно.
- Мають метод swap() та відповідне перевантаження std::swap().

- Мають метод empty() — повертає true для порожніх.
- Мають метод maxsize() — повертає максимальний теоретичний розмір контейнера.
- (Крім forward_list з C++11 — у ньому відсутній для оптимізації), мають метод size().
- (Крім array, який має фіксований розмір), мають наступні метод clear() — повне очищення[2].
- (Крім array, який має фіксований розмір), мають методи insert() – вставка та erase() — видалення елементів (хоча їх сигнатури у різних контейнерів можуть відрізнятися). C++11: також метод emplace(). forward_list має свої назви для цих методів: insert_after, erase_after, emplace_after.
- C++20 додав незалежні перевантаження функцій std::erase() та std::erase_if() для видалення елементів із не-асоціативних контейнерів[3]. Вони, зокрема, позбавляють необхідності використовувати ідіому remove-erase для вектора.
- До асоціативних контейнерів C++20 додав метод contains().

[2] Однак, зазвичай, без зміни розміру виділеного блоку. Див. такоє есей "Does ‘v.clear()‘ free the allocation?"

[3] https://quuxplusone.github.io/blog/2020/07/08/erase-if/ – цікава стаття на тему.

Типи, оголошені в контейнерах

Всі контейнери (X) оголошують наступні типи (typedef):

- X::value_type
- X::reference
- X::const_reference
- X::iterator
- X::const_iterator
- X::reverse_iterator (крім forward_list)
- X::const_reverse_iterator (крім forward_list)
- X::difference_type
- X::size_type
- X::pointer
- X::const_pointer
- X::allocator_type (крім array)

Послідовні контейнери/послідовності (Sequence Containers)

- Послідовності — впорядковані контейнери, такі, про які для кожного елемента можна сказати, який іде перед ним, і який після.
- До них належать vector, deque, list, array (C++11), forward_list – з парою обмовок (C++11).
- Всі вони (крім array) мають конструктори виду[4]:

```
S::S(S::size_type n, const S::value_type& t);
S::S(S::const_iterator first, S::const_iterator last);
```

та аналогічні методи assign():

```
S::assign(S::size_type n, const S::value_type& t);
S::assign(S::const_iterator first, S::const_iterator last);
```

- Не всюди нижче вказано це явно (щоб не загромаджувати текст), але array не має жодного методу, що може змінювати розмір.

[4] C++11 також додає конструктори та assign() із списками ініціалізації.

- Крім array та forward_list, мають методи insert виду:

```
S:: insert (S:: iterator  before, const S::value_type& val); // 1
S:: insert (S:: iterator  before, S::size_type nVal,
                  const S::value_type& val); // 2
S:: insert (S:: iterator  before, S:: const_iterator  first ,
                  S:: const_iterator  last ); // 3
```

  C++11 додало до них варіант, що працює із списком ініціалізації та із rvalue reference (для використання переміщення).

- Всі вони вставляють елементи **перед** тим, на який вказує before. 1 — один елемент, 2 — nVal копій елемента, 3 — інтервал.
- array просто незмінний, а forward_list надає аналогічні варіанти цього методу, з іменем insert_after(), які вставляють елементи **після** елемента, на який вказує ітератор.
- Швидкодія – лінійна для vector і deque, постійна для list та forward_list відносно розміру контейнера, у всіх випадках лінійна відносно кількості елементів, що вставляються. (Детальніше див. документацію).
- Повертають валідний ітератор на перший із нових елементів. (В C++98 методи, які вставляють декілька елементів, не повертають нічого). Для vector інвалідують всі ітератори, починаючи із місця, де відбулася вставка, якщо відбулася реалокація — інвалідуються взагалі всі, deque інвалідує тільки ітератори після. Для list і forward_list не інвалідують жодного ітератора. Те ж стосується посилань на елементи.

- Крім array та forward_list, мають методи erase виду:

```
S::iterator  S::erase(S::iterator  position );  // 1
S::iterator  S::erase(S::const_iterator  first ,
                      S::const_iterator  last );  // 2
```

  яким передається ітератор на елемент, який слід стерти, або інтервал, який починається ітератором на елемент, що слід стерти, і закінчується ітератором на перший, який слід залишити.
- Повертають ітератор на перший залишений елемент (може бути рівним end()).
- array просто незмінний, а forward_list надає аналогічні варіанти цього методу, з іменем erase_after(), які видаляють елементи **після** елемента, на який вказує ітератор.
- Для vector і deque складність лінійна по розміру контейнера, для списків — постійна.
- Для vector інвалідує ітератори від точки видалення і далі, для deque інвалідує всі ітератори (крім видалення з початку або з кінця – тоді тільки ітератори на елементи, що видаляються), для списків — інвалідуються лише ітератори на елементи, які видаляються. Те ж стосується посилань на елементи.

- Всі *послідовні* контейнери мають метод front(), який повертає посилання на перший елемент, і всі, крім forward_list, мають back(), який повертає посилання на останній.
- Всі, крім vector (та array), мають метод push_front() — вставити на початок. Для deque інвалідує всі ітератори, включаючи end(), але **НЕ** інвалідує посилань на елементи, для списків не інвалідує жодного ітератора чи посилання.
- Всі, крім forward_list (та array), мають push_back() — додати елемент в кінці. Для вектора — якщо не відбулося реалокації, інвалідується лише end(), інакше інвалідуються всі ітератори та посилання. Для deque інвалідує всі ітератори, включаючи end(), але **НЕ** інвалідує посилань на елементи, для list не інвалідує жодного ітератора чи посилання.
- У тих випадках, коли існують push_back()/push_front(), C++ додав emplace_back()/emplace_front() із такою ж поведінкою та таким же впливом на валідність ітераторів, які, однак, конструюють об'єкти, які вставляють, безпосередньо на місці — позбавляючи необхідності робити копії.

- Всі послідовні, крім vector (та array), мають метод pop_front(), а всі, крім forward_list (та array) — pop_back(), які відкидають, відповідно, перший чи останній елементи послідовності. Не повертають нічого із міркувань безпеки щодо виключень. Не інвалідують ітераторів та посилань, крім тих, що вказують на елемент, який видаляється, та, можливо, для vector і deque – end() (див. документацію за подробицями).
- Всі, крім array, мають метод resize(), котрий змінює розмір (за збільшення може заповнювати нові елементи переданим значенням). Деталі інвалідації мали б бути інтуїтивно зрозумілі.
- Послідовні контейнери-не списки, array, vector, deque, мають можливість звертання до елемента за індексом — оператор [], який не зобов'язаний контролювати вихід за межі контейнера, та метод at(idx), котрий кидає виключення out_of_range у такому випадку.

vector

vector детально розглядався в окремій лекції, тому тут тільки коротко.

- Послідовний контейнер, із довільним доступом.
- Дозволяє легко додавати елементи в кінець, та відкидати з кінця.
- Характерною особливістю є неперервність в пам'яті. Завдяки цьому, його можна використовувати при роботі із legacy-кодом, який потребує вбудованих масивів. Крім того, ітерація по ньому може бути дуже швидкою (на відміну від того ж deque).
- Щоб одночасно забезпечити швидке додавання в кінець та неперервність в пам'яті, резервує її більше, ніж безпосередньо потрібно. Після вичерпання резерву, резервує вдвічі більше — відбувається реалокація. Сам ніколи не звільняє зайву пам'ять, але є способи його змусити(див. відповідну лекцію).
- Через можливість реалокації під час збільшення розміру, правила інвалідації ітераторів мають винятки, виду "не інвалідує <....>, якщо не відбулася реалокація, інакше інвалідує всі ітератори та посилання".

array — C++11

- Трішки нетиповий контейнер — не може змінювати свій розмір. Фактично, є тонкою обгорткою над вбудованими масивами C, не використовує динамічної пам'яті, тому економний і ефективний.
- Час swap для нього лінійний (на відміну від інших контейнерів).
- Конструктор по замовчуванню — автоматично генерований, тому елементи вбудованих типів залишаються неініціалізованими (на відміну від інших контейнерів).
- Дозволяє довільний доступ. Має стандартний комплект засобів, див. вище — [], at(), back(), front().
- Має метод fill(), який заповнює його переданим значенням.
- Надає інтерфейс доступу до елементів під час компіляції, реалізований в дусі tuple (std::tuple_size<std::array>, std::tuple_element<std::array>). Про tuple див. далі.
- Деякі подробиці щодо реалізації: "Inside STL: The array".
- "std::array in C++ isn't slower than array in C". А іноді буває й швидшим – наприклад, через гарантовану відсутність aliasing-гу: "std::array in C++ is faster than array in C. Sometimes".

```
std :: array<double,5> arr;

arr . fill (3.1415);

for ( double& x : arr){
    std :: cout << x << ", ";
}
std :: cout << endl;

for( size_t i=0; i<arr.size (); ++i ){
    std :: cout << arr[i] << ", ";
}
std :: cout << endl;

for( auto i=arr.begin (); i!=arr.end (); ++i ){
    std :: cout << *i << ", ";
}
std :: cout << endl;
```

```
for( size_t i=0; i<arr.size(); ++i ){
    arr[i] /= i+1;
}

for ( double& x : arr){
    std::cout << x << ", ";
}
std::cout << endl;

sort(arr.begin(), arr.end());

for ( double& x : arr){
    std::cout << x << ", ";
}
std::cout << endl;
```

Двохстороння черга — std::deque

- deque — **d**ouble-**e**nded **que**ue, двостороння черга.
- Контейнер, з точки зору користувача — дуже схожий на vector.
- Основна відмінність — дозволяє легко додавати елементи як в кінці, так і на початку.
- Щоб реалізувати цю можливість, на відміну від вектора, дані не обов'язково розташовані в пам'яті неперервно. Зазвичай реалізується як набір сегментів — впорядкованих між собою масивів фіксованого розміру.
- Ріст з кінців швидший, ніж для вектора, тому що ніколи не потрібна реалокація.
- Однак, через сегментованість, доступ до елементів може бути повільнішим, ніж для vector.
- Не має спеціалізації для bool, тому може використовуватися як заміна vector<bool>.
- Дозволяє звертатися до елементів за індексом, як оператором [], так і at().
- Набір методів розглядався вище: size(), resize(), empty(), clear(), push_back(), pop_back(), push_front(), pop_front(), swap(), front(), back(), begin(), end(), тощо[5]
- Додавання елементів спереду зсуває індекси всіх інших!
- C++11 Додав метод shrink_to_fit(), для мінімізації використаної пам'яті, хоча для deque він менш критичний, ніж для vector (та й deque має право оптимізувати використання пам'яті на свій розсуд, в принципі, як і вектор).

[5] Взагалі, якщо певний метод має сенс для певних контейнерів, він зазвичай буде присутнім, і називатиметься так само.

Приклад 1

```
deque<int> d1;

const int rmax = 20;

for(int i=0; i<100; ++i){
    int t = (rand()%rmax - rmax/2);
    if(t>=0){
        d1.push_back(t);
    }else{
        d1.push_front(t);
    }
}

for(size_t i = 0; i<d1.size(); ++i){
    cout << d1[i] << ", ";
}
cout << endl << endl;
```

Приклад 2 — використання ітераторів

```
for(deque<int>::iterator itr = d1.begin(); itr !=d1.end(); ++itr )
{
    cout << *itr << ", ";
}
```

Приклад 3 — видаляємо всі нулі

erase(), як і інші операції вставки чи видалення, псує — інвалідує, ітератори.

Щоб справитися із цим, erase() та insert() повертають вказівник, на наступний після видаленого або на перший вставлений, відповідно, елемент.

Наприклад, щоб видалити із черги нулі, можна зробити так:

```
for(deque<int>::iterator itr = d1.begin(); itr !=d1.end();){
    if( *itr == 0){
        itr = d1.erase( itr ); // Продовжуємо з наступного
    }else{
        ++itr;
    }
}
```

Хоча, звичайно, краще скористатися відповідним алгоритмом, remove.

Посилання

- https://devblogs.microsoft.com/oldnewthing/20230809-00/?p=108577 – "Inside STL: The deque, design".
- https://devblogs.microsoft.com/oldnewthing/20230810-00/?p=108587 – "Inside STL: The deque, implementation".

Списки — std::list

- std::list є послідовним контейнером, однак помітно відмінним відмінним від vector чи deque. Його основна задача – вставка та видалення елементів за константний час. Тому типова реалізація – двонаправлений динамічний список.
- При тому, довільного доступу до елементів нема — доступні лише Bidirectional ітератори. Єдиний прямий доступ до елементів надають методи front() і back()
- Методи, size(), resize(), empty(), clear(), push_back(), pop_back(), push_front(), pop_front(), front(), back(), swap(), begin(), end(), та інші функціюють, на загал, так само, як і для вектора.
- Так само, вище описано поведінку методів insert(), єдине, що вони не інвалідують ітераторів чи посилань.
- Однак, через технічні причини, size() для списку може мати лінійну складність, а не константну, як для інших контейнерів. Подробиці — див. далі.
- Модифікації list не псують ітераторів, крім тих, які вказують на елементи, що видаляються!

Специфічні методи list — 1

- splice() переміщає елементи з одного списку в інший. Може переміщати інтервали в межах одного списку.
  - Якраз через slice(), size() може потребувати не константний час. Справа в тому, що неможливо зробити константним одночасно і size() і splice() — один із них мусить бути лінійним. Тому до C++11 це було залишено на розсуд розробників стандартної бібліотеки — різні її реалізації могли вибирати, котрий зробити константним, size() чи splice(). Починаючи із C++11 вимагається, щоб константним був саме size() — узгоджено із іншими контейнерами.

  Детальніше про нього далі.
- merge() — в припущенні, що два списки посортовані, об'єднує їх в один, теж посортований. Існує два варіанти: l1.merge(l2), l1.merge(l2,comparator). Вважає, що списки посортовані за зростанням. Перший використовує для порівняння оператор <, другий — переданий предикат comparator(). l2 стає порожнім. Операція стабільна — відносний порядок еквівалентних елементів не змінюється.

Специфічні методи list — 2

Велика кількість алгоритмів вимагає ітераторів довільного доступу — той же sort буде дуже важко зробити ефективним із Bidirectional ітераторами[6]. Тому list надає ряд методів, аналогічних стандартним алгоритмам, ефективних саме для цього контейнера:

- l.remove (val) — видаляє ВСІ елементи із значенням val.
- l.remove_if(pred) — видаляє всі елементи, для яких предикат pred(elem) повертає true.
- l.erase (pos) — видаляє елемент в положенні pos, і повертає ітератор на наступний елемент.
- l.unique() і c.unique(pred2) — видаляє послідовні однакові за значенням елементи. Перший варіант використовує для порівняння оператор ==, друга — бінарний предикат pred2(elem1, elem2).
- l.reverse() — змінює порядок елементів на зворотній.
- l.sort(), l.sort(pred2) — сортує по зростанню. Перший варіант користується оператором <, другий — бінарним предикатом pred2

[6] Не маючи зараз посилань на повноцінні дослідження питання, наведу лише аналогію. З двосторонніми ітераторами легко реалізувати хіба сортування бульбашкою — для швидкого сортування їх недостатньо.

splice

Нехай l1, l2 — два списки, pos — ітератор із l1, куди відбувається вставка.

- l1.splice(pos,l2) — **перемістити** всі елементи з l2 в l1 перед pos. Після цього l2 стає порожнім. l1 і l2 не можуть співпадати. Час – константний.
- l1.splice(pos,l2,l2pos) — перемістити один елемент з l2 у положенні l2pos в список, l1, перед pos. l1 та l2 можуть співпадати. Час – константний.
- l1.splice(pos,l2,l2begpos, l2endpos) — перемістити елементи із інтервалу [l2begpos,l2endpos) в l1, перед pos. l1 і l2 можуть співпадати, однак, pos повинен бути за межами інтервалу [l2begpos,l2endpos). Час — до лінійного (див. також вище про size()).

C++11 додає move-варіанти цих методів.

Приклад — 1

```
list <char> d1,d2;

string  inpstr1("abcdefgh"),  inpstr2("12345464748");

for(size_t i=0;i<inpstr1.length(); ++i){
    d1.push_back(inpstr1[i]);
}
for(size_t i=0;i<inpstr2.length(); ++i){
    d2.push_back(inpstr2[i]);
}

cout << "d1 = ";
for( list <char>::iterator itr = d1.begin(); itr !=d1.end(); ++itr){
    cout << *itr << ", ";
}
cout << endl << endl;
cout << "d2 = ";
for( list <char>::iterator itr = d2.begin(); itr !=d2.end(); ++itr){
    cout << *itr << ", ";
}
cout << endl << endl;
```

Приклад — 2

Вставляємо один список в інший, після четвертого елемента.
Зверніть увагу на використання advance — ітератор типу `list<char>::iterator` є ітератором послідовного доступу:

```
list <char>::iterator  itr2  = d1.begin();
advance( itr2 ,4);
d1. splice ( itr2 ,d2);
```

Видаляємо всі '4':

```
d1.remove('4' );
```

Приклад — 3

Спробуємо видалити всі елементи, які є літерами, використовуючи функцію isalpha. Однак, її прототип:

```
int  isalpha (int );
```

a remove_if для list<char> очікуватиме функцію, яка приймає (строго) char. Як вже обговорювалося, для того, щоб обійти проблему, або можна написати свій **адаптер**:

```
class  MyFunctionAdaptor{
    typedef int  (*Fn_Ptr)(int);  // Прототип функцій isXXX з cctype
                                  // Щоб можна було легко використо-
вувати
                                  // як аргумент, вжито typedef
    Fn_Ptr fn_ptr;
public:
    MyFunctionAdaptor(Fn_Ptr in_fn_ptr): fn_ptr(in_fn_ptr){}
    bool operator()(char c) const {
        return fn_ptr(c);
    }
};
```

Тоді:

```
d1.remove_if( MyFunctionAdaptor(isalpha) );
```

Приклад — 3.1

Або, альтернативний підхід — використати стандартний шаблон unary_function з файлу <functional>:

```
template <class Arg, class  Result>
    struct  unary_function {
            typedef Arg argument_type;
            typedef Result  result_type;
        };
```

Наприклад, ось так:

```
#include <functional>

template <class T>
struct  isalpha_adapter  :  public  std :: unary_function<T, bool>{
     bool operator()  (T c) const {
          return  isalpha (c);
     }
};

d1.remove_if(isalpha_adapter<char>() );
```

Також, можна написати свій шаблон-адаптер, який прийматиме як параметр функцію, яка викликатиметься.

forward_list — однозв'язний список — C++11

- Як і std::list — контейнер із швидкою вставкою та видаленням, яке досягається ціною відсутності довільного доступу. На відміну від list, підтримує лише рух вперед — має лише Forward ітератори. Якщо таких ітераторів достатньо — є ефективнішим, зокрема – з точки зору використання пам'яті, ніж list. Реалізується однозв'язним списком.
- Як і для list, маніпуляції над forward_list не інвалідують ітератори та посилання, крім тих, що вказують на елементи, які видаляються.
- Нетиповим для контейнерів є те, що не містить методу size(). Це зроблено для максимальної ефективності — реалізація цього методу привела б до помітних накладних витрат. Якщо розмір таки потрібен, можна зробити так: distance(fl.begin(), fl.end()), який зразу робить очевидним лінійність обчислення розміру по часу.

- Так як повернутися назад неможливо, вставка та видалення відбуваються із елементами *після* поточного, тому, на відміну від list, називаються ці методи із суфіксом _after.
- insert_after() — працює, як і описані вище методи insert() для послідовних контейнерів, тільки додає елементи *після* pos.
- erase_after() — те ж.
- Однак, "завдяки" додаванню чи видаленню після вказаного елемента, їх не можна використовувати для роботи із першим елементом списку. Тому для цього є ряд спеціалізованих методів:
- push_front() додає елемент на початку.
- emplace_front() — аналогічно, але конструює його безпосередньо на місці.
- pop_front() — відкидає перший елемент.
- pop_front() не кидає виключень, решта перерахованих дають строгу гарантію.

- merge() поводиться так само, як і для list.
- splice_after() – те саме, але вставляє *після* вказаного елемента.
- remove(), remove_if(), reverse(), unique(), sort() аналогічні до тих, що в list. Безпеку щодо виключень і складність, див. документацію, наприклад `http://en.cppreference.com/w/cpp/container/forward_list` або `http://www.cplusplus.com/reference/forward_list/forward_list/`.

Асоціативні контейнери

- Крім послідовних контейнерів, стандартна бібліотека включає так звані асоціативні контейнери. Формально — це реалізація поняття асоціативного масиву.
- В C++98 є 4 асоціативних *впорядкованих* контейнери — set, multiset, map, multimap.
- C++11 додала ще 4 асоціативних *невпорядкованих*, або *хешованих* контейнери — unordered_set, unordered_multiset, unordered_map, unordered_multimap.
- Контейнери ці зберігають *пари* ключ–значення. Строго кажучи, це стосується тільки відображень, map і т.д., Однак, для простоти можна вважати, що для set і компанії, що ключ і значення співпадають[7].

[7] Зрозуміло, що в *set* фізично зберігається тільки ключ, який зразу служить і значенням — ніякого надмірного дублювання немає. :-)

- Послідовні асоціативні контейнери, (C++11, boost, сторонні бібліотеки), зберігають свої елементи впорядковано. Для впорядкування використовується оператор $<$ (або переданий компаратор). Зазвичай вони реалізовуються із використанням збалансованих дерев (Self-balancing binary search tree). Час доступу до елементів та час вставки/видалення — логарифмічний по розміру, O(log N). (Рух ітераторів теж логарифмічний в загальному випадку, однак якщо послідовно застосовувати лише інкремент чи лише декремент — амортизований константний).
- Невпорядковані асоціативні контейнери зберігають свої елементи із використанням хеш-таблиць. Вимагають, щоб для елементів було реалізовано оператор ==. Час доступу, середній — константний (!), найгірший випадок – лінійний по кількості елементів. На практиці, зазвичай – помітно швидші за впорядковані асоціативні контейнери, але використовують більше пам'яті. Див., також, наприклад: `http://bannalia.blogspot.com/2013/10/implementation-of-c-unordered.html` — "mplementation of C++ unordered associative containers".

Послідовні асоціативні контейнери

- Володіють комплектом операцій, характерних і для інших контейнерів: begin(), end(), rbegin(), rend(), (їх константні варіанти в C++11: cbegin(), cend(), crbegin(), crend()), size(), max_size, empty(), clear(), swap().
- Присутні також звичні для послідовних контейнерів методи erase(), insert(), emplace(), однак їх використання дещо відрізняється, детальніше див. далі.
- count() повертає кількість елементів з ключем, рівних переданому. Може повертати 0 або 1 для set/map, інші додатні значення для multiset/multimap.
- find() повертає ітератор на елемент із ключем, еквівалентним заданому, або end(), якщо елемента із таким ключем немає. Для multiset/multimap повертає ітератор на перший елемент із еквівалентним ключем.

- equal_range() повертає пару ітераторів, яка задає інтервал, де знаходяться елементи із ключем, еквівалентним переданому:

  pair<iterator , iterator > equal_range
  (**const** key_type& key) **const**;

  (Нагадаємо, що ключ і значення для set/multiset рівні). Якщо такого елемента немає, повертається інтервал, довжиною нуль — обидва ітератори співпадають, вказуючи на елемент, який йшов би *після* того, який шукається, згідно внутрішнього впорядкування контейнера, якби такий існував.
- lower_bound() повертає ітератор на перший із елементів, з еквівалентним до переданого ключем, або, якщо такого немає, ітератор на елемент, що йшов би *після* того, який шукається, згідно внутрішнього впорядкування контейнера, якби такий існував.
- upper_bound() повертає ітератор на елемент, який май би йти зразу після елементів із заданим ключем.

Впорядкування та еквівалентність елементів

- Типи, які служать ключами (зразу і елементами для set/multiset), повинні бути впорядковуваними згідно заданого критерію. По замовчуванню для впорядкування використовується less<>, котрий використовує оператор <. Можна передавати користувацький предикат — компаратор.
- Еквівалентність елементів визначається співвідношенням: !(a<b) && !(b<a) — ні a не менше за b, ні b не менше за a.
- Порівняння елементів повинно задовольняти вимоги слабкого строгого впорядкування (strict weak ordering):
  - Асиметричність: якщо x < y істинне, то y < x не істинне.
  - Транзитивність: якщо x < y і y < z, повинно виконуватися x < z.
  - Не-рефлексивність: x < x завжди не істинне.
  - Транзитивність еквівалентності: якщо x еквівалентний y, а y еквівалентний z, то x еквівалентний z. Зокрема, ця вимога забороняє використання операторів виду <= в ролі компаратора.
- Метод key_comp() повертає об'єкт, який використовується в ролі компаратора ключів.
- value_comp() повертає компаратор значень. Для set/multiset, через рівність ключів та значень, еквівалентний до того, що повертає key_comp().

Видалення елементів

```
void erase ( iterator position ); \\ [1]
size_type erase (const key_type& k); \\ [2]
void erase ( iterator first , iterator last ); \\ [3]
```

- Не інвалідують ітераторів чи посилань на елементи, крім тих, що видаляють.
- Варіант [2] повертає кількість видалених елементів.
- Гарантії щодо виключень: якщо компаратор не кидає виключень — теж ніколи не кидає. Якщо ж він може кидати виключення, але видаляється один елемент — строга гарантія, інакше — базова.
- Складність: [1] — константна, [2] — логарифмічна, [3] — лінійна по довжині інтервалу.

Вставка елементів — 1

Для set/map:

```
pair<iterator,bool> insert (const value_type& val); // [1a]
```

Для multiset/multimap:

```
iterator insert (const value_type& val); // [1b]
```

Спільні:

```
iterator insert ( iterator position , const value_type& val); // [2]
template <class InputIterator>
    void insert ( InputIterator first , InputIterator last ); // [3]
```

- [1a] вставляє елемент. Так як більше одного елемента із еквівалентним ключем в set/map не може бути, повертає пару, в котрій другий елемент — булівське значення, рівне true, якщо вставка відбулася, false, якщо еквівалентний елемент/ключ вже присутній в контейнері, а перший — ітератор на вставлений (чи вже присутній) елемент.
- [1b] — вставляє елемент. Так як multiset/multimap може містити довільну кількість елементів, вставка завжди успішна, тому повертає лише ітератор на вставлений елемент.

Вставка елементів — 2

- [2] — вставляє елемент, користуючись переданим ітератором як підказкою — для оптимізації вставки[8]. Тобто, це тільки рекомендація!).
- [3] — вставляє інтервал елементів.
- Вставка одного елемента дає строгу гарантію, вставка багатьох за раз — базову.
- Складність — логарифмічна, однак, якщо передано вдалу підказку (варіант [2]), амортизована константна.

[8] Пошук місця для елементу почнеться з з переданого ітератора, position. Кінцеве положення буде завжди тим же, але вдало підібрані підказки можуть сильно прискорити процес.

Вставка елементів — 3 — C++11

- C++11 додав emplace() та emplace_hint(), які додають елементи, конструюючи їх безпосередньо на місці, другий із них — з підказкою, де елемент розташувати.
- emplace() для set/map повертає пару, як і insert(), всі решта — ітератор на новий (чи еквівалентний вже існуючий) елемент.

Для set/map:

```
template <class... Args>
  pair<iterator,bool> emplace (Args&&... args);
template <class... Args>
  iterator emplace_hint (const_iterator position, Args&&... args);
```

Для multiset/multimap:

```
template <class... Args>
  iterator emplace (Args&&... args);
template <class... Args>
  iterator emplace_hint (const_iterator position, Args&&... args);
```

Додаткові типи даних

- key_type — тип ключа (для set/multiset співпадає із value_type).
- key_compare, value_compare — типи відповідних компараторів.



- std::set — приклад асоціативного контейнера.
- Він зберігає множину елементів. Кожен елемент має бути унікальним.
- Внутрішнє представлення є впорядкованим, подробиці див. вище.
- std::multiset — множина, що може містити декілька однакових елементів.
- Доступ до елементів та вставка, в загальному випадку, вимагають логарифмічного часу.
- Модифікації set/multiset не псують ітераторів, крім тих, що вказують на елементи, що видаляються.
- set і multiset оголошені в <**set**>.

```
template < class T,                          // set :: key_type/value_type
           class Compare = less<T>,          // set::key_compare/value_compare
           class Alloc = allocator<T>        // set::allocator_type
           > class set;

template < class T,                          // multiset :: key_type/value_type
           class Compare = less<T>,          // multiset::key_compare/value_com
           class Alloc = allocator<T> >      // multiset::allocator_type
           > class multiset;
```

Пам'ятка щодо std::set/std::multiset

- Різні варіанти insert()/emplace() — основний спосіб додати елемент. Всілякі push_back() для асоціативного контейнера позбавлені сенсу. Повертає ітератор на новий елемент та чи була вставка успішною (pair<iterator,bool>) для set (на противагу multiset). insert(elem, pos) — оптимізована версія, яка використовує переданий ітератор pos як підказку, де (приблизно) вставляти.
- Варіанти erase() — видалити елементи із контейнера.
- find(elem) — повертає ітератор на elem, або end(), lower_bound() та upper_bound() повертають верхню та нижню межу для заданого елемента в контейнері.
- count (elem) — підраховує кількість елементів, повертаючи 0 або 1 для set, довільне значення для multiset.
- Через сортування по ключах, які зразу є й елементами, на льоту ключ змінювати не можна. Єдиний спосіб — видалити елемент, змінити і додати наново.
- Детальніше — див. документацію, наприклад: `http://www.cplusplus.com/reference/set/set/`.

Splicing – C++17

- В C++17 додано можливість переміщати вузол із set/map, multiset/multimap та їх unordered-варіантів ,в інший – без копіювання самих даних.

```
set<string> src, dest;

src.insert("ABC");
    .................
auto handle = src.extract(std::string("ABC"));
// src.extract("ABC") теж можна, але тут це менш наочно.
dest.insert(std::move(handle));
```

- extract() інвалідує ітератори на елемент, що переміщається. Вказівники та посилання залишаються валідними, але їх не можна використовувати, поки елемент не буде додано до іншого контейнера.
- Якщо такого елемента немає, повертає порожній дескриптор вузол.
- Існує в двох варіантах:

```
node_type extract( const_iterator position );
node_type extract( const key_type& x );
```

- Кожен контейнер містить відповідний тип node_type – тип дескриптора вузла.
- extract(), як і erase() може спричиняти перебалансування контейнера.

Об'єднання асоціативних контейнерів – merge – C++17

- Новий метод merge() всіх [multi|unordered_][set|map] дозволяє перемістити всі елементи з одного контейнера в інший.

```
std :: set<std:: string > src,  dest;
    ..................
dest.merge(src);

for  (auto& elem : dest)
     std :: cout  << elem << '\n';
std :: cout  << "----------------" << std::endl;
for  (auto& elem : src)
     std :: cout  << elem << '\n';
```

- Алгоритмічна складність – N*log(size()+N)), де N – src.size().
- Алокатори обох контейнерів повинні співпадати.

Приклад

```
set<string> present;

present.insert("Ivan");        present.insert("Maria");
present.insert("Andriy");      present.insert("Svitlana");
present.insert("Stiv");        present.insert("Zeleboba");

cout << "Present: ";
for(set<string>:: iterator itr = present.begin();
                           itr!=present.end(); ++ itr){
    cout << *itr << ", ";
}
cout << endl << endl;

string ask_for("Olena");
if( present.find(ask_for)!=present.end() ){
    cout << ask_for << " is present" << endl;
}else{
    cout << ask_for << " is absent" << endl;
}
```

std::map/std::multimap — відображення

- std::map — ще один асоціативний контейнер, власне — асоціативний масив.
- Зберігає пари ключ-значення. Дозволяє створювати сутності, які ведуть себе ніби масиви із індексом практично довільного типу — був би для нього означеним operator<() та операції копіювання.
- Зокрема, його ітератор вказує на пару типу pair<KeyT, ValT>.
- Ключі в map не повинні повторюватися. Існує також multimap, який дозволяє однакові ключі.
- Через сортування по ключу, на льоту ключ змінювати не можна. Єдиний спосіб — видалити елемент із тим ключем і додати наново. Змінювати самі елементи можна довільно.
- Модифікації map не псують ітераторів, крім тих, що вказують на елементи, що видаляються.

<map>

- Оголошені map та multimap в <map>

```
template < class Key,                          // map::key_type
           class  T,                           // map::mapped_type
           class  Compare = less<Key>, // map::key_compare
           class  Alloc  =  allocator<pair<const Key,T> >
                                               // map::allocator_type
           > class  map;

template < class Key,                          // multimap::key_type
           class  T,                           // multimap::mapped_type
           class  Compare = less<Key>, // multimap::key_compare
           class  Alloc  =  allocator<pair<const Key,T> >
                                               // multimap::allocator_type
           > class  multimap;
```

Пам'ятка щодо std::map/std::multimap

- insert(elem) — вставляє пару ключ-значення, elem є чимось типу make_pair(key, val). Інші версії insert() поводяться аналогічно.
- erase(key) — видалити всі елементи із ключем key, повертає кількість видалених. Інші версії erase() отримують ітератор/інтервал, поводяться так, як і для інших контейнерів.
- find(key) — повертає положення елемента із ключем key, або end().
- count (key) — підраховує кількість елементів з ключем key.
- Детальніше — див. документацію, наприклад: `http://www.cplusplus.com/reference/set/set/`.

map::operator[]

- Унікальний метод map — перевизначений оператор індексації, operator[](key):

  mapped_type& **operator**[] (**const** key_type& k);

- Оператор цей працює для std::map дещо незвично. При його виклику, якщо елемента із таким ключем відсутній, він додається до map, із значенням по замовчуванню.
- Тобто, константного варіанту цього методу немає.
- Для мозахістів, виклик цього оператора еквівалентний до:

  (*((**this**->insert(make_pair(k,mapped_type()))).first)).second

- C++11 додала методі map::at():

  mapped_type& at (**const** key_type& k);
  const mapped_type& at (**const** key_type& k) **const**;

  який кидає виключення out_of_range, якщо елемента із таким ключем не знайдено.

Приклад map::operator[]

```
std :: map<char,int> m1;
m1['A'] = 100;
m1['B'] = 200;

int a = m1['A'];
int b = m1['C'];
```

- Якщо елемент із вказаним ключем існує, буде повернено посилання на нього.
- Якщо елемента з таким ключем **не існує**, його буде додано, а значенню присвоєно його значення по замовчуванню. Тобто, у прикладі вище "невинний" код `int b = m1['C'];` додасть елемент з ключем 'C', рівний нулю.

const map&

Через цю особливість з константними об'єктами його використовувати не можна, доводиться викручуватися:

```
void print_if_present(const std::map<char,int> &m, char el)
{
    // Звертання m[...] заборонене!
    // Воно може модифікувати відображення.
    std::map<char,int>::const_iterator itr = m.find(el);
    if( itr!=m.end() )
    {
        cout << "Element " << el << " is present, equalt to "
             << itr->second << endl;
             // itr вказує на паруб pair<char, int> -- ключзначення-.
    }else{
        cout << "Element " << el << " is not present" << endl;
    }
}
```

Приклад — рахунки клієнтів

```
map<string, double> account;
account["Ivan"] = 300;
account["Maria"] = -0.30;
account["Andriy"] = -100;
account["Svitlana"] = 222;
account["Stiv"] = 111;

cout << left << "Debtors: \n";
for(map<string, double>::iterator itr = account.begin();
        itr !=account.end(); ++itr)
{
    if ( itr ->second < 0)
    {
        cout << setw(20) << itr->first << ":";
        cout << setw (10) << itr->second << endl;
    }

}
```

Гетерогенний пошук (heterogeneous lookup) – C++14

- Приклад:

```
std::map<std::string, int> data = {....};
if (data.find("Text") != intMap.end()){ .... }
```

Цей код неефективний, так як, хоча std::string та C-стрічку можна порівнювати безпосередньо, компаратор map, по замовчуванню – less<...> цього не вміє, тому буде створено тимчасову std::string (з динамічною алокацією і все таке).

- Причина:

```
std::map<std::string, int>
```

є еквівалентним до:

```
std::map<std::string, int, std::less<std::string>,
                    std::allocator<std::pair<const std::string, int>>>
```

Так як оголошено його так:

```
template<
    class Key,                                  class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T>>
> class map;
```

- Та ж проблема буде і в інших предикатів та інших функторів – std::greater, std::plus і т.д.
- В C++14 додали таку спеціалізацію less<>:

```
template <>
struct less<void> {
    using is_transparent = int;

    // Код спрощено для наочності
    template <class T, class U>
    constexpr auto operator()(T&& lhs, U&& rhs) const
        return static_cast<T&&>(lhs) < static_cast<U&&>(rhs);
    }
};
```

На додачу до попередньої:

```
template <class T = void>
struct less {
    constexpr bool operator()(const T& lhs, const U& rhs) const {
        return lhs < rhs;
    }
};
```

- Метод пошуку перевантажено так:

```
template <class _Other, class _Mycomp = key_compare,
          class = typename _Mycomp::is_transparent>
 iterator  find(const _Other& _Keyval) { ... }
```

- Якщо less<> має typedef (байдуже який) is_transparent – цей метод і буде використано, якщо немає більш точного.
- Тоді, для гетерогенного пошуку можна робити так:

```
std::map<std::string, int, std::less<>> data = {.....};
if (data.find("Text") != intMap.end()){ .... }
```

- Той же трюк можна використати для пошуку за частиною ключа – наприклад, Id, абощо. (Тільки ключ, за яким шукається, має збігатися з ключем, яким посортовано контейнер!)
- Однак, ні з operator[], ні з методом .at() це не працює – string_view не можна неявно перетворювати у std::string – для запобігання зайвим неочевидним копіюванням, а відповідних перевантажень ці методи не мають аж до C++26.
- Детальніше див. `https://www.bfilipek.com/2019/05/heterogeneous-lookup-cpp14.html`.

- C++20 додав гетерогенний пошук для unordered-контейнерів.
  - Однак, її використання потребує більше зусиль, див., наприклад "C++20: Heterogeneous Lookup in (Un)ordered Containers": потрібно реалізувати компаратор та хеш-функцію для всіх залучених типів.
  - Приклад звідти та з цього тексту:

```
struct string_hash {
  // Вказує що потрібно використати прозоре хешування
  using is_transparent = void;
  // Хешування повинно бути узгодженим:
  // a == b => hash(a) == hash(b)
  size_t operator()(const char *txt) const {
      return std::hash<std::string_view>{}(txt);
  }
  size_t operator()(std::string_view txt) const {
      return std::hash<std::string_view>{}(txt);
  }
  size_t operator()(const std::string &txt) const {
      return std::hash<std::string>{}(txt);
  }
};

// std::equal_to -- використовуємо оператор == для std::string,
//  він вже має потрібні перевантаження
std::unordered_map<std::string, std::string,
        string_hash, std::equal_to<>> string_map;
```

- C++23 додав підтримку гетерогенного пошуку до erase() та extract().

C++17

- extract() та merge() було описано вище, в розділі про set – вони дуже схожі.
- Стандарт C++17 додав відображенням також методи try_emplace() і insert_or_assign().
- try_emplace() є, фактично, заміною emplace(). Різниця в тому, що він не переміщатиме move-only аргументи, якщо вставки не відбулося і працює із ключем та значенням, а не створеною із них парою:

```
std::map<int, std::string> m;
m.try_emplace(17, "abc"s);
```

- Щоправда, гетерогенний пошук із try_emplace() не працює.
- insert_or_assign() вставляє новий елемент, якщо елемента із таким ключем ще не було, заміняє значення, якщо ключ існує.

```
std::map<int, std::string> m;
m.insert_or_assign(17, "xyz"s);
```

- На відміну від operator[], вміє працювати і з типами без конструктора за замовчуванням. На відміну від insert() – працює, навіть коли елемента в контейнері немає.
- C++20 також додав метод contains().

Невпорядковані/хешовані контейнери

- C++11 додав чотири нових асоціативних контейнери: unordered_set, unordered_multiset, unordered_map, unordered_multimap. Перших два оголошені в <unordered_set>, других два в <unordered_map>.
- За інтерфейсом вони дуже сході на "старі" асоціативні контейнери із C++98, однак реалізовані за допомогою хеш-таблиць[9].
- Через те, що не включення цих контейнерів в C++98 виглядало дивно — вони дуже корисні, і для них існували готові реалізації, багато розробників компіляторів включили їх як нестандартні розширення, з іменами hash_XXX. Тому, коли розроблявся TR1, а потім C++11, їх включили під іменами unordered_XXX — щоб уникнути конфліктів.
- Основна перевага — вони забезпечують, в середньому, лінійний час доступу до елементів. (Хоча, константа ця і більша, ніж для інших контейнерів із константним доступом, так як відбувається обчислення хеш-функції. Тому, зокрема, робота із інтервалами може бути повільнішою, ніж для впорядкованих асоціативних контейнерів). Ціною цього є більші витрати пам'яті.
- C++20 додав до цих класів .contains() та підтримку heterogeneous lookup.

[9] http://en.wikipedia.org/wiki/Hash_table

Приклад використання гетерогенного пошуку

```
struct string_hash {
    using is_transparent = void;
    [[nodiscard]] size_t operator()(const char *txt) const {
        return std::hash<std::string_view>{}(txt);
    }
    [[nodiscard]] size_t operator()(std::string_view txt) const {
        return std::hash<std::string_view>{}(txt);
    }
    [[nodiscard]] size_t operator()(const std::string &txt) const {
        return std::hash<std::string>{}(txt);
    }
};
std::unordered_map<std::string, res_data_t, string_hash,
                                        std::equal_to<>> data;
```

Взято з "C++20: Heterogeneous Lookup in (Un)ordered Containers"

- C++26 нарешті додав повну підтримку гетерогенного пошуку: "Extending associative containers with the remaining heterogeneous overloads".

- Раймонд Чен навів цікаву табличку[10]:

Дія	**Спосіб**
Read, throw if missing	`m.at(key)`
Read, allow missing	`m.find(key)`
Read, create if missing	`m[key]`
Write, no-op if exists, discard value	`m.insert({ key, value })`, `m.emplace(key, value)`
Write, nop if exists	`m.emplace(std::piecewise_construct, ...)`, `m.try_emplace(key, params)`
Write, overwrite if exists	`m.insert_or_assign(key, value)`

- В C++29 до всіх різновидів map додали `.lookup(key)`, який повертає std::optional не має проблеми `operator[]`, спричинених додаванням відсутніх елементів та find, який знаходить лише елементи, що вже існують.

[10] "The operations for reading and writing single elements for C++ standard library maps". Див. також його наступний есей: "A simplified overview of ways to add or update elements in a std::map".

Посилання

- `http://en.cppreference.com/w/cpp/container`
- `http://en.cppreference.com/w/cpp/concept/UnorderedAssociativeContainer`
- `https://habr.com/ru/post/660265/` – "Особенности реализации STL в Clang, GCC и Microsoft C++", трохи про подробиці реалізації хешованих структур даних, оптимізацій в алгоритмах і т.д.
- "Inside STL: The map, set, multimap, and multiset".
- "Inside STL: The unordered_map, unordered_set, unordered_multimap, and unordered_multiset".
- "How do I add a non-copyable, non-movable object to a std::map or std::unordered_map?".
- Цікава функція хешування: "Better Bit Mixing - Improving on MurmurHash3's 64-bit Finalizer".
- "How useful is the hint passed to the std::unordered_... collections?"
- "‘map::operator[]‘ should be nodiscard".

flat_XXX (C++23) (DRAFT)

- В C++23 додали контейнери flat_set, flat_map, flat_multiset, flat_multimap, змодельовані згідно аналогічних з Boost.Container (`https://www.boost.org/doc/libs/1_82_0/doc/html/container.html`).
- Вони надають інтерфейс аналогічний відповідним контейнерам, розглянутий раніше, але зберігають дані в послідовностях. За замовчуванням використовують std::vector.
- Пошук у таких контейнерах швидкий, зокрема – через малу константу, але вставка чи видалення – повільні. Тобто, це спеціалізовані засоби.
- Подробиці див., наприклад, тут: `https://www.boost.org/doc/libs/1_82_0/doc/html/boost/container/flat_set.html`, `https://www.boost.org/doc/libs/1_82_0/doc/html/boost/container/flat_map.html`.

Особливості flat_XXX

- Зацитовано за: `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1222r0.pdf`.
- Faster lookup than standard associative containers.
- Much faster iteration than standard associative containers.
- Random-access iterators instead of bidirectional iterators.
- Less memory consumption for each element.
- Improved cache performance (data is stored in contiguous memory).
- Non-stable iterators (iterators are invalidated when inserting and erasing elements).
- Non-copyable and non-movable values types can't be stored.
- Weaker exception safety than standard associative containers (copy/move constructors can throw when shifting values in erasures and insertions).
- Slower insertion and erasure than standard associative containers (specially for non-movable types).

Сторонні асоціативні контейнери

- Тест великої кількості асоціативних контейнерів: "An Extensive Benchmark of C and C++ Hash Tables"
  - ◇ На сучасних процесорах std::unordered_map відверто програє новим контейнерам.
- "Inside boost::unordered_flat_map"
  - ◇ Станом на червень 2024, сторінка документації boost::unordered_flat_map підвішує браузери, хоча вона ніби не така вже й велика. Вони навчилися ламати не тільки компілятори хитрозакрученими шаблонами...

Адаптери

- stack — адаптер, який працює як стек. Надає операції додати елемент до вершини (push()), подивитися його (top()), викинути (pop()). (Детальніше див. лекцію про vector). Повний список операцій: empty(), size(), back(), push, pop(), emplace() (C++11), плюс традиційні конструктор, оператори співвідношень, swap() (C++11).
- queue — одностороння черга, додавати елементи можна тільки з кінця (push_back()), видаляти — тільки з початку (pop_front()). Повний список операцій: empty(), size(), front(), back(), push(), pop(), emplace() (C++11), плюс традиційні конструктор, оператори співвідношень, swap() (C++11).
- priority_queue — черга з пріоритетом. Додавати можна тільки з кінця, забирати з початку, при чому, першим елементом завжди буде той, що має найвищий пріоритет[11]. Пріоритети порівнюються згідно оператора <, або переданим компаратором. Повний список операцій: empty(), size(), top(), push(), pop(), emplace() (C++11), плюс традиційні конструктор та swap() (C++11).

stack оголошений в <stack>, queue і priority_queue в <queue>.

[11] back() контейнера, який адаптує priority_queue відповідатиме вершині, top(), черги.

std::inplace_vector – C++26

- Запропонований в C++26 std::inplace_vector, динамічний вектор з capacity, фіксованою під час компіляції.
  - ◇ Моделювався за "boost::static_vector".

C++26

- Запропонований в C++26 std::hive, високопродуктивну структуру даних, яка допускає часте додавання та видалення елементів, без інвалідації вказівників та ітераторів.
  - Інші назви таких структур: “bucket array” та “object pool”.
  - Часто такі структури потрібні в іграх, під час моделювання багаточастинкових систем, у високочастотному трейдингу, робототехніці тощо.
  - std::list для цього підходить погано, через погану локальність у пам’яті.
  - Є блоковою структурою (схожа до std::deque ідея), яка підтримує skipfield – список видалених елементів, місце яких може використовуватися повторно.
  - Додавання елементу до повного контейнеру, як і видалення, не спричиняє переміщення елементів.
  - Ітератори залишаються валідними, не зважаючи на додавання та видалення.
  - Може зберігати об’єкти, які не можна переміщати чи копіювати.
  - Невпорядкований, bidirectional.
  - Можна отримати ітератор з вказівника на елемент.

- `https://habr.com/ru/post/664044/` – "Неклассические контейнеры в C++"

- TODO

- TODO
- Про небезпеку поліморфних алокаторів: "How frivolous use of polymorphic allocators can imbitter your life"

